Hardening a contact form against abuse

> **Belona Cybersecurity** · October 2026 Real case: the contact form on `belonacyber.com` . Stack: Cloudflare Workers · Turnstile · Resend. *Versión en español:* `espanol/endurecimiento-del-formulario.md`

The problem

A form that sends email **is an open relay if it isn't protected**. That isn't rhetoric — it's the precise technical description of an endpoint that accepts text from anyone and turns it into outbound mail.

The damage isn't junk in the inbox. It's this chain:

```
a bot finds the endpoint
   → sends spam with YOUR domain as the sender
      → receivers mark that domain as a spam source
         → your sending reputation drops
            → your legitimate mail starts landing in spam
```

That last step is what ruins a business built on writing to prospects. **You lose exactly what SPF, DKIM and DMARC were protecting** — through a door you opened yourself.

That's why this form has eight controls. Not paranoia: the failure mode is losing the ability to communicate.

The eight layers

Outside in. Each one rejects as cheaply as possible, before spending resources on the next.

1 · Minimal surface

The Worker serves **one endpoint**: `POST /api/contact` . Everything else goes straight to static assets without running any code of ours.

```
if (url.pathname !== '/api/contact') return env.ASSETS.fetch(request);
if (request.method !== 'POST') return json({ ok:false, error:'method_not_allowed' }, 405);
```

2 · Origin

The submission must come from the site itself. A request from another domain is rejected before the body is read.

```
const origen = request.headers.get('origin');
if (!origen || !ORIGENES_VALIDOS.includes(origen)) return json({...}, 403);
```

An honest limitation: the `Origin` header is set by the browser, not the user — but a tool like `curl` can forge it. This layer stops abuse from other websites, **not a determined attacker**. It's here because it's free, not because it's sufficient.

3 · Size cap before parsing

```
if (Number(request.headers.get('content-length') || 0) > 8 * 1024)
  return json({ ok:false, error:'too_large' }, 413);
```

Rejected **before** the body is parsed as JSON. Parsing first and validating afterwards is the mistake that turns a form into a resource-exhaustion vector.

4 · Captcha verified server-side

```
const r = await fetch('https://challenges.cloudflare.com/turnstile/v0/siteverify',
  { method:'POST', body: form });
return (await r.json()).success === true;
```

The point isn't having a captcha — it's where it's checked. The browser is under the attacker's control. A client claiming "I passed the captcha" is worth nothing. The only answer that counts is Cloudflare's, requested from the server with the secret key.

A captcha validated client-side is decorative.

5 · Per-field validation and caps

```
name    ≤ 120    and at least 2 characters
email   ≤ 254    (the real maximum for an address) and well-formed
company ≤ 160
message ≤ 4000   and at least 20 characters
```

Email validation is deliberately simple. **The only real proof that an address exists is sending something to it**; an exhaustive regular expression produces false negatives on valid addresses and adds no security.

6 · Honeypot

A field named `website`, off-screen and out of the tab order:

```
<div class="hp" aria-hidden="true">
  <input name="website" type="text" tabindex="-1" autocomplete="off">
</div>
```

```
.hp { position:absolute; left:-9999px; width:1px; height:1px; overflow:hidden }
```

A person never sees it or reaches it. Many bots fill in everything they find in the DOM.

`display:none` **is deliberately avoided**: a good share of bots skip anything hidden that way. Moving it off-screen keeps it present for whoever reads the markup and absent for whoever uses the page.

And the decision that actually matters:

```
if (limpiar(datos.website, 50) !== '') return json({ ok: true });
```

It answers `ok`, not an error. Returning an error would teach the bot that this field gives it away, and it would stop filling it. Returning success sends it away believing the submission worked.

> Deceiving an attacker is worth more than blocking one loudly.

7 · Secrets never touch disk

`TURNSTILE_SECRET` and `RESEND_API_KEY` are loaded with `wrangler secret put`: they travel from the terminal to Cloudflare's encrypted store **without being written to any file**. There is no `.env`.

Not even an authenticated session can read them back:

```
$ npx wrangler secret list
[ { "name": "RESEND_API_KEY",   "type": "secret_text" },
  { "name": "TURNSTILE_SECRET", "type": "secret_text" } ]
```

Names only. `.gitignore` covers `.env`, `.dev.vars` and similar as **defense in depth** — none should exist, but if a tool ever created one, git won't publish it.

The sending key was created with **Sending access**, not *Full access*: the Worker only needs to send mail. With full permissions, that same leaked key would allow deleting the verified domain or minting more keys.

8 · Asymmetric errors

```
const detalle = await envio.text();
console.error(JSON.stringify({ evento:'fallo_envio', estado: envio.status,
                               respuesta: detalle.slice(0,600) }));
return json({ ok:false, error:'send_failed' }, 502);
```

The visitor is told it failed. The logs are told why.

A detailed error facing outward is free reconnaissance: it reveals the provider, the version, sometimes the internal structure. An opaque message with no internal logging is worse still — nobody can diagnose anything.

The verification

All eight layers were exercised **against the live production site**, with `curl`. They aren't assumed to work because the code says so:

TEST	RESPONSE
`GET` to the endpoint	`405 method_not_allowed`
`POST` with no `Origin` header	`403 bad_origin`
`POST` from a different origin	`403 bad_origin`
5-character message	`400 mensaje`
Malformed email address	`400 correo`
Honeypot filled in	`200 {"ok":true}` **— nothing sent**
No captcha token	`403 captcha`
9 KB body	`413 too_large`

This table is the deliverable. Anyone can claim controls are in place; what separates security work from a list of good intentions is evidence that each one was exercised and answered as designed.

What this setup does NOT cover

Saying so is part of the work. A report that only lists what it does isn't a report — it's advertising.

- **No per-IP rate limiting.** Turnstile stops cheap automation, but an attacker who solves captchas can submit repeatedly. Pending mitigation: a Cloudflare rate-limiting rule, or a counter in KV.

- **The `Origin` header can be forged** by tools outside the browser. It stops cross-site abuse, not a determined attacker.
- **No malware scanning or content analysis** on the submitted text. The message is escaped before being embedded in the email HTML, which closes injection, but the content itself isn't inspected.
- **The visitor's email address isn't verified.** It goes into `reply_to` without checking that it exists. Verifying it would require a confirmation step that would sharply reduce legitimate submissions — a deliberate trade-off, not an oversight.

Why it's written this way

The goal wasn't to add controls, but **to make every one of them defensible**. A control nobody can justify is the first one someone removes when it gets in the way.

And the failure mode driving the whole design isn't "I get spam." It's **"I lose the ability to reach my clients"** — the one that actually costs money.

From the technical portfolio of Belona Cybersecurity. The full source lives in the site's private repository; the snippets here are verbatim.